

Limitation of Code first approach using EF in [ASP.NET Core](#)

1. Difficult with Large Existing Databases :

- a. Not suitable when the organization already has a huge production database with hundreds of tables and stored procedures

2. Migration Complexity :

- a. Managing migrations becomes difficult in large enterprise applications with multiple developers

3. Risk of Data Loss :

- a. Incorrect migration execution may affect production data

4. Limited Database Control :

- a. DBAs may not prefer developers directly modifying schema through code

5. Complex SQL Features Handling :

- a. Difficult to implement advanced DB features like triggers, partitioning, and complex stored procedures

6. Performance Tuning Challenges :

- a. Database optimization often requires manual SQL tuning beyond EF capabilities

7. Synchronization Issues :

- a. Keeping models and database synchronized across environments can become difficult

In what scenario do we rely on the DB first approach ?

Scenario	Why DB First is Preferred
Legacy Enterprise Systems	Existing databases already contain business-critical data
DBA-Controlled Organizations	Database schema managed by database

	administrators
Large Enterprise Applications	Complex schemas with views, triggers, SPs, functions
Banking & Insurance Applications	Strict database governance and auditing requirements
Database Shared Across Multiple Applications	Schema changes cannot be controlled by a single application
Existing Stored Procedure Heavy Systems	Easier integration with existing SQL logic
Reverse Engineering Requirement	Generate models directly from existing database

Problem Statement

A company named SmartMart Retail already has an existing SQL Server database for managing products and customers.

The development team needs to build an ASP.NET Core MVC application using the existing database without creating tables manually from code.

The application should:

- **Connect with the existing SQL Server database**
- **Automatically generate models from database tables**

- Perform CRUD operations
- Reduce manual coding effort
- Quickly start development using Visual Studio

Getting started with Database First approach (Steps)

Step	Process
Step 1	Create Database in SQL Server
Step 2	Create ASP.NET Core Project
Step 3	Install Entity Framework Packages
Step 4	Configure Connection String
Step 5	Scaffold Database
Step 6	Generate Models & DbContext
Step 7	Perform CRUD Operations

Final Architecture Flow

```
Browser Request
    ↓
MVC Controller
    ↓
Entity Framework Core
    ↓
DbContext
    ↓
SQL Server Database
```

Why is the Model first approach not followed in Enterprise project development ?

Reason	Explanation
Limited Flexibility	Difficult to handle frequent enterprise-level schema changes

Poor Scalability	Not ideal for large applications with evolving requirements
Complex Maintenance	Managing visual models becomes difficult over time
Version Control Issues	Diagram/model files create merge conflicts in team environments
Limited Support in EF Core	Full Model First support is not available in modern EF Core
Less Preferred in Agile Development	Agile teams prefer faster code-centric development
Difficult Integration with Existing DB	Not practical when databases already exist

What is the repository pattern ? while developing ASP.NET Core

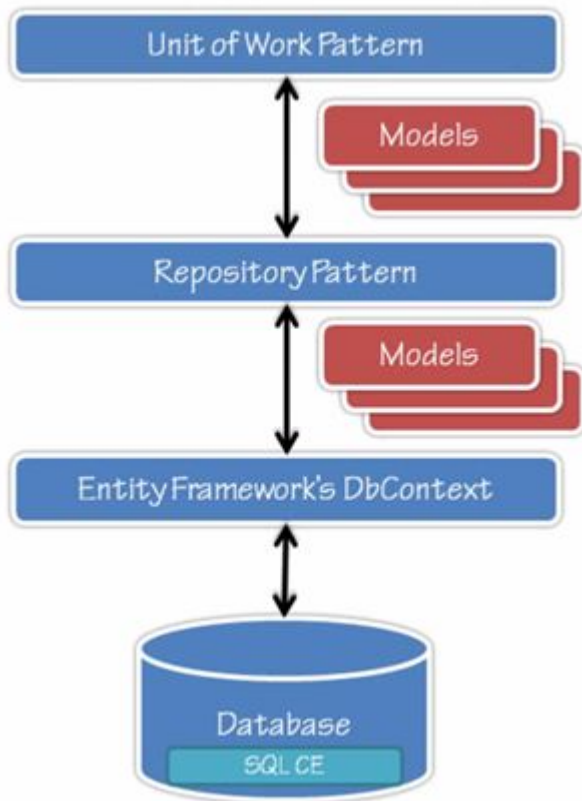
The **Repository Pattern** is a software design pattern that acts as an abstraction layer between the application's business logic and data access logic.

It helps separate:

- Business Logic
- Database Access Logic

application, why is it useful for enterprise development ?

Benefit	Explanation
Separation of Concerns	Keeps business logic separate from database logic
Better Maintainability	Easier to manage and modify code
Reusability	Common CRUD operations can be reused
Unit Testing Support	Easier to mock repositories during testing
Reduced Code Duplication	Centralized data access implementation
Improved Scalability	Supports layered enterprise architecture
Database Independence	Easier to switch database providers
Cleaner Controllers	Controllers remain lightweight and focused



Scenario

An e-commerce ASP.NET Core application has:

- Product Module
- Order Module
- Customer Module
- Payment Module

Without Repository Pattern:

- Each controller directly communicates with Entity Framework.
- Code duplication increases.
- Testing becomes difficult.

With Repository Pattern:

- Data access logic is centralized.

- Controllers call repository methods instead of direct EF queries.
- Easier maintenance and scalability.

What is AJAX, implementation of AJAX in ASP.NET core ?

AJAX (Asynchronous JavaScript and XML) is a web development technique used to send and receive data from the server asynchronously without refreshing the entire webpage.

Advantage	Explanation
Faster User Experience	Partial page updates
Reduced Server Load	Only required data transferred
Better Responsiveness	No full page reload
Improved Performance	Faster interaction with server
Dynamic UI Updates	Real-time updates possible

What is used as an extension of AJAX in [ASP.NET](#) application development ?

Technology	Purpose
jQuery AJAX	Simplifies AJAX calls
Fetch API	Modern JavaScript-based asynchronous requests
Axios	Promise-based HTTP client
SignalR	Real-time communication in ASP.NET Core
Partial Views	Dynamically update sections of UI
JSON	Standard data exchange format
Web API	RESTful service communication

